

Project 2: Java Spring Boot Inventory API CI/CD Demo

Overview

This project demonstrates a standard CI/CD pipeline using Jenkins, Maven, SonarQube, Docker, and Trivy for a Java Spring Boot application. It highlights Maven builds, Surefire test reports, and fat-JAR packaging.

Prerequisites

Ensure the following are installed and running on your Jenkins server:

- Docker
- Trivy
- Java 17 (JDK)
- Maven

Step 1: Network Setup

Create a shared Docker network for your infrastructure and deployments:

```
docker network create jenkins-net
```

Step 2: Infrastructure Setup

Run SonarQube and Jenkins on the `jenkins-net` network:

```
# Run SonarQube
```

```
docker run -d --name sonarqube -p 9000:9000 --network jenkins-net  
sonarqube:lts-community
```

```
# Run Jenkins (with Docker socket mounted)
```

```
docker run -d --name jenkins -p 8081:8080 -p 50000:50000 --network  
jenkins-net -v /var/run/docker.sock:/var/run/docker.sock -v  
jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

(Note: If Jenkins is running on 8080 already, you can keep it. The app exposes 8080, so map Jenkins to a different port like 8081 if running on the same host, or change the app port.)

Step 3: Jenkins Configuration

1. Log into Jenkins.
2. Install necessary plugins: Docker Pipeline, SonarQube Scanner, Pipeline, JUnit Plugin.
3. Configure SonarQube Server:

- Go to Manage Jenkins -> System.
 - Add a SonarQube server named SonarQube and point it to <http://sonarqube:9000>.
4. Configure Global Tools:
- Go to Manage Jenkins -> Tools.
 - Add Maven and JDK installations if not relying on system defaults.

Step 4: Pipeline Execution

1. Create a new Pipeline job in Jenkins.
2. Choose "Pipeline script from SCM" or paste the contents of Jenkinsfile.
3. Run the build.

Step 5: Verification

Once the pipeline finishes successfully, access the deployed Spring Boot API:

- URL: <http://localhost:8080/inventory>
- Expected Output: `{"item":"Laptop","quantity":50,"status":"In Stock"}`